FOUNDATION OF JAVA LAB SHEET – 9

Collections

Objective: To understand the basic concepts of variable, decision and loop control statements.
Requirements:

VS Code editor or eclipse or IntelliJ Idea IDE
JDK 19 or higher

Q1. Student List Create an ArrayList containing ten student names. Display all names and the total size.

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> students = new ArrayList<>();

        students.add("Sumit");
        students.add("Rahul");
        students.add("Aman");
        students.add("Rohit");
        students.add("Karan");
        students.add("Arjun");
        students.add("Vikas");
        students.add("Mohit");
        students.add("Ankit");
        students.add("Neeraj");

        for (String name : students)
            System.out.println(name);

        System.out.println("Total Students: " + students.size());
    }
}
```

OUTPUT:
Sumit
Rahul
Aman
Rohit
Karan

Arjun
Vikas
Mohit
Ankit
Neeraj
Total Students: 10

Q2. Marks List Create an ArrayList containing five marks. Display the first, last and middle elements using get().

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> marks = new ArrayList<>();

        marks.add(70);
        marks.add(80);
        marks.add(90);
        marks.add(85);
        marks.add(95);

        System.out.println("First: " + marks.get(0));
        System.out.println("Middle: " + marks.get(2));
        System.out.println("Last: " + marks.get(4));
    }
}
```

OUTPUT:
First: 70
Middle: 90
Last: 95

Q3. Update Student Name Create an ArrayList. Replace the second student name using set() and remove the fourth student.

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
```

```
        ArrayList<String> students = new ArrayList<>();

        students.add("Sumit");
        students.add("Rahul");
        students.add("Aman");
        students.add("Rohit");
        students.add("Karan");

        students.set(1, "Arjun");  // Replace 2nd name
        students.remove(3);        // Remove 4th student

        System.out.println(students);
    }
}
```

OUTPUT:
[Sumit, Arjun, Aman, Karan]

Q4. Unique Departments Create a HashSet containing department names with duplicates. Display the resulting unique departments.

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        HashSet<String> departments = new HashSet<>();

        departments.add("IT");
        departments.add("HR");
        departments.add("Finance");
        departments.add("IT");
        departments.add("HR");

        System.out.println("Unique Departments: " + departments);
    }
}
```

OUTPUT:
Unique Departments: [Finance, HR, IT]

Q5. Sorted Roll Numbers Create a TreeSet containing ten roll numbers in random order. Display them in sorted order.

```
import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        TreeSet<Integer> rolls = new TreeSet<>();

        rolls.add(105);
        rolls.add(101);
        rolls.add(110);
        rolls.add(103);
        rolls.add(108);
        rolls.add(102);
        rolls.add(107);
        rolls.add(104);
        rolls.add(109);
        rolls.add(106);

        System.out.println("Sorted Roll Numbers: " + rolls);
    }
}
```

OUTPUT:
Sorted Roll Numbers: [101, 102, 103, 104, 105, 106, 107, 108, 109, 110]

Q6. ArrayList Searching Create an ArrayList of course names. Ask the user for a course and use contains() to check whether it exists.

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> courses = new ArrayList<>();

        courses.add("BCA");
        courses.add("BBA");
        courses.add("B.Tech");
        courses.add("MCA");
```

```
        courses.add("MBA");

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter course: ");
        String course = sc.nextLine();

        if (courses.contains(course))
            System.out.println("Course Found");
        else
            System.out.println("Course Not Found");
    }
}
```

OUTPUT:
Enter course: BCA
Course Found

Q7. Collections Utility Methods Create an ArrayList and demonstrate Collections.sort(), reverse(), max(), min() and frequency().

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();

        numbers.add(10);
        numbers.add(30);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);

        Collections.sort(numbers);
        System.out.println("Sorted: " + numbers);

        Collections.reverse(numbers);
        System.out.println("Reverse: " + numbers);

        System.out.println("Maximum: " + Collections.max(numbers));
        System.out.println("Minimum: " + Collections.min(numbers));
        System.out.println("Frequency of 30: " +
                        Collections.frequency(numbers, 30));
```

```
    }
}
```

OUTPUT:
Sorted: [10, 20, 30, 30, 40]
Reverse: [40, 30, 30, 20, 10]
Maximum: 40
Minimum: 10
Frequency of 30: 2

Q8. LinkedList Student Queue Create a LinkedList representing students waiting for a lab. Use addFirst(), addLast(), removeFirst() and removeLast().

```
import java.util.LinkedList;

public class Main {
    public static void main(String[] args) {
        LinkedList<String> students = new LinkedList<>();

        students.addFirst("Sumit");
        students.addLast("Rahul");
        students.addLast("Aman");
        students.addFirst("Rohit");

        System.out.println("Queue: " + students);

        students.removeFirst();
        students.removeLast();

        System.out.println("After Removal: " + students);
    }
}
```

OUTPUT:
Queue: [Rohit, Sumit, Rahul, Aman]
After Removal: [Sumit, Rahul]

Q9. ListIterator Modification Create a List and use ListIterator to traverse it. Use set() to replace one element and add() to insert an element during iteration.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();

        names.add("Sumit");
        names.add("Rahul");
        names.add("Aman");

        ListIterator<String> it = names.listIterator();

        while (it.hasNext()) {
            String name = it.next();

            if (name.equals("Rahul"))
                it.set("Arjun");

            if (name.equals("Aman"))
                it.add("Rohit");
        }

        System.out.println("Updated List: " + names);
    }
}
```

OUTPUT:
Updated List: [Sumit, Arjun, Aman, Rohit]

Q10. List vs Set Experiment Create an ArrayList and HashSet using the same input values. Display both and explain the difference in duplicates and ordering.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] values = {10, 20, 10, 30, 20, 40};

        ArrayList<Integer> list = new ArrayList<>();
        HashSet<Integer> set = new HashSet<>();
```

```
        for (int n : values) {
            list.add(n);
            set.add(n);
        }

        System.out.println("ArrayList: " + list);
        System.out.println("HashSet: " + set);
    }
}
```

OUTPUT:
ArrayList: [10, 20, 10, 30, 20, 40]
HashSet: [10, 20, 30, 40]

Q11. Sorted Student IDs Create a TreeSet for student IDs. Add duplicate IDs and display the unique sorted IDs. Also demonstrate first(), last(), higher() and lower().

```
import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        TreeSet<Integer> ids = new TreeSet<>();

        ids.add(105);
        ids.add(101);
        ids.add(103);
        ids.add(101);
        ids.add(108);
        ids.add(105);

        System.out.println("Sorted IDs: " + ids);
        System.out.println("First: " + ids.first());
        System.out.println("Last: " + ids.last());
        System.out.println("Higher than 103: " + ids.higher(103));
        System.out.println("Lower than 103: " + ids.lower(103));
    }
}
```

OUTPUT:
Sorted IDs: [101, 103, 105, 108]
First: 101
Last: 108

Higher than 103: 105
Lower than 103: 101

Q12. Hospital Patient List Create an ArrayList of patient names and a HashSet of unique patient IDs. Add, remove, search and display the records.

```
import java.util.*;

public class Main {
   public static void main(String[] args) {
      ArrayList<String> patients = new ArrayList<>();
      HashSet<String> ids = new HashSet<>();

      // Add records
      patients.add("Sumit");
      patients.add("Rahul");
      patients.add("Aman");

      ids.add("P101");
      ids.add("P102");
      ids.add("P103");

      // Remove
      patients.remove("Rahul");
      ids.remove("P102");

      // Search
      System.out.println("Sumit exists: " + patients.contains("Sumit"));
      System.out.println("P103 exists: " + ids.contains("P103"));

      // Display
      System.out.println("Patients: " + patients);
      System.out.println("Patient IDs: " + ids);
   }
}
```

OUTPUT:
Sumit exists: true
P103 exists: true
Patients: [Sumit, Aman]
Patient IDs: [P101, P103]

Q13. ListIterator Report Create an ArrayList of five course names. Use ListIterator to display the list forward and backward, and demonstrate nextIndex() and previousIndex().

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> courses = new ArrayList<>();

        courses.add("Java");
        courses.add("Python");
        courses.add("C++");
        courses.add("HTML");
        courses.add("SQL");

        ListIterator<String> it = courses.listIterator();

        System.out.println("Forward:");
        while (it.hasNext()) {
            System.out.println(it.next());
        }

        System.out.println("Backward:");
        while (it.hasPrevious()) {
            System.out.println(it.previous());
        }

        it.next();
        System.out.println("Next Index: " + it.nextIndex());
        System.out.println("Previous Index: " + it.previousIndex());
    }
}
```

OUTPUT:

```
Forward:
Java
Python
C++
HTML
SQL

Backward:
SQL
```

HTML
C++
Python
Java

Next Index: 1
Previous Index: 0

Q14. ArrayList vs LinkedList Lab Create equivalent ArrayList and LinkedList programs. Perform insertion, deletion, get(), size() and iteration operations. Record your observations in a comparison table.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        ArrayList<String> a = new ArrayList<>();
        LinkedList<String> l = new LinkedList<>();

        // Insertion
        a.add("Java");
        a.add("Python");
        a.add("C++");

        l.add("Java");
        l.add("Python");
        l.add("C++");

        // Deletion
        a.remove("Python");
        l.remove("Python");

        // get()
        System.out.println("ArrayList get: " + a.get(1));
        System.out.println("LinkedList get: " + l.get(1));

        // size()
        System.out.println("ArrayList size: " + a.size());
        System.out.println("LinkedList size: " + l.size());

        // Iteration
```

```
        System.out.println("ArrayList: " + a);
        System.out.println("LinkedList: " + l);
    }
}
```

OUTPUT:
ArrayList get: C++
LinkedList get: C++
ArrayList size: 2
LinkedList size: 2
ArrayList: [Java, C++]
LinkedList: [Java, C++]

Q315 Mini Student Collection Management System Create a student collection management program. Use ArrayList for maintaining student records in insertion order, HashSet for unique student IDs, and TreeSet for sorted student IDs. Provide options to add, remove, search, display, count, and iterate students. Use ListIterator for forward/backward traversal where appropriate. Demonstrate the difference between List, Set and SortedSet.

```
import java.util.*;

public class Main {
    static ArrayList<String> students = new ArrayList<>();
    static HashSet<Integer> uniqueIds = new HashSet<>();
    static TreeSet<Integer> sortedIds = new TreeSet<>();

    static void addStudent(int id, String name) {
        students.add(name);
        uniqueIds.add(id);
        sortedIds.add(id);
    }

    static void removeStudent(String name, int id) {
        students.remove(name);
        uniqueIds.remove(id);
        sortedIds.remove(id);
    }

    static void searchStudent(String name) {
        if (students.contains(name))
            System.out.println("Student Found");
```

```
        else
            System.out.println("Student Not Found");
    }

    static void display() {
        System.out.println("Students: " + students);
        System.out.println("Unique IDs: " + uniqueIds);
        System.out.println("Sorted IDs: " + sortedIds);
        System.out.println("Count: " + students.size());
    }

    static void iterate() {
        ListIterator<String> it = students.listIterator();

        System.out.println("Forward:");
        while (it.hasNext())
            System.out.println(it.next());

        System.out.println("Backward:");
        while (it.hasPrevious())
            System.out.println(it.previous());
    }

    public static void main(String[] args) {

        addStudent(103, "Aman");
        addStudent(101, "Sumit");
        addStudent(102, "Rahul");
        addStudent(101, "Rohit"); // Duplicate ID

        display();

        System.out.println("\nSearch:");
        searchStudent("Sumit");

        System.out.println("\nRemove:");
        removeStudent("Rahul", 102);
        display();

        System.out.println("\nIteration:");
        iterate();
    }
}
```

OUTPUT:
Students: [Aman, Sumit, Rahul, Rohit]
Unique IDs: [101, 102, 103]
Sorted IDs: [101, 102, 103]
Count: 4

Search:
Student Found

Remove:
Students: [Aman, Sumit, Rohit]
Unique IDs: [101, 103]
Sorted IDs: [101, 103]
Count: 3

Iteration:
Forward:
Aman
Sumit
Rohit
Backward:
Rohit
Sumit
Aman